Advanced Matrix Concepts for Machine Learning and Deep Learning

December 16, 2025

Contents

1 Minor and Cofactor **3**
- 1.1 Introduction . . . 3
- 1.2 Minor of a Matrix . . . 3
- 1.3 Cofactor of a Matrix . . . 4
- 1.4 Computing Determinant using Cofactor Expansion . . . 5
- 1.5 Applications in Machine Learning . . . 6

2 Adjoint of a Matrix **7**
- 2.1 Definition and Concept . . . 7
- 2.2 Step-by-Step Computation . . . 7
- 2.3 Adjoint for 2×2 Matrices . . . 8
- 2.4 Applications in Machine Learning . . . 9

3 Inverse of a Matrix **11**
- 3.1 Definition and Properties . . . 11
- 3.2 Methods for Computing Inverse . . . 11
  - 3.2.1 Method 1: Using Adjoint (Analytical) . . . 11
  - 3.2.2 Method 2: Gauss-Jordan Elimination . . . 12
  - 3.2.3 Method 3: For 2×2 Matrices (Direct Formula) . . . 13
- 3.3 Pseudo-Inverse (Moore-Penrose Inverse) . . . 13
- 3.4 Applications in Machine Learning and Deep Learning . . . 15
- 3.5 Computational Considerations . . . 16

4 Rank of a Matrix **17**
- 4.1 Definition and Concepts . . . 17
- 4.2 Methods for Computing Rank . . . 17
  - 4.2.1 Method 1: Row Echelon Form (REF) . . . 17
  - 4.2.2 Method 2: Determinants of Submatrices . . . 18
  - 4.2.3 Method 3: Singular Value Decomposition (SVD) . . . 18
- 4.3 Applications in Machine Learning . . . 19
- 4.4 Rank-Nullity Theorem . . . 21

5 Column Space and Null Space **22**
5.1 Column Space (Range) 22
5.2 Null Space (Kernel) 23
5.3 Geometric Interpretation 24
5.4 Applications in Machine Learning 25

6 Change of Basis **27**
6.1 Introduction and Motivation 27
6.2 Change of Basis Matrix 27
6.3 Matrix Representation in Different Bases 28
6.4 Orthonormal Bases 28
6.5 Applications in Machine Learning and Deep Learning 30
6.6 Summary and Key Takeaways 33

7 Conclusion **33**
7.1 Practical Recommendations 34
7.2 Further Study 34

1 Minor and Cofactor

1.1 Introduction

Minor and cofactor are fundamental concepts in linear algebra that are essential for computing determinants and matrix inverses. While they may not be directly used in high-level machine learning algorithms, they form the theoretical foundation for many computational techniques in ML/DL.

1.2 Minor of a Matrix

Definition 1.1 (Minor)**.** Let $\mathbf{A}$ be an $n \times n$ square matrix. The minor M_{ij} of element a_{ij} is the determinant of the $(n-1) \times (n-1)$ submatrix obtained by deleting the i-th row and j-th column from $\mathbf{A}$.

Mathematically, for matrix $\mathbf{A} = [a_{ij}]_{n \times n}$:

$$M_{ij} = \det(\mathbf{A}_{-i,-j}) \tag{1}$$

where $\mathbf{A}_{-i,-j}$ denotes the matrix $\mathbf{A}$ with row i and column j removed.

Example 1.1 (Computing Minors for a 3×3 Matrix)**.** Consider the matrix:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \tag{2}$$

Step 1: Compute the minor M_{11} (remove row 1, column 1):

$$\mathbf{A}_{-1,-1} = \begin{bmatrix} 4 & 5 \\ 2 & 1 \end{bmatrix} \tag{3}$$

$$M_{11} = \det \begin{bmatrix} 4 & 5 \\ 2 & 1 \end{bmatrix} = (4)(1) - (5)(2) = 4 - 10 = -6 \tag{4}$$

Step 2: Compute the minor M_{12} (remove row 1, column 2):

$$\mathbf{A}_{-1,-2} = \begin{bmatrix} 1 & 5 \\ 3 & 1 \end{bmatrix} \tag{5}$$

$$M_{12} = \det \begin{bmatrix} 1 & 5 \\ 3 & 1 \end{bmatrix} = (1)(1) - (5)(3) = 1 - 15 = -14 \tag{6}$$

Step 3: Compute the minor M_{13} (remove row 1, column 3):

$$\mathbf{A}_{-1,-3} = \begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix} \tag{7}$$

$$M_{13} = \det \begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix} = (1)(2) - (4)(3) = 2 - 12 = -10 \tag{8}$$

Step 4: Similarly, compute remaining minors:

$$M_{21} = \det \begin{bmatrix} 3 & 1 \\ 2 & 1 \end{bmatrix} = 3 - 2 = 1 \tag{9}$$

$$M_{22} = \det \begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix} = 2 - 3 = -1 \tag{10}$$

$$M_{23} = \det \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix} = 4 - 9 = -5 \tag{11}$$

$$M_{31} = \det \begin{bmatrix} 3 & 1 \\ 4 & 5 \end{bmatrix} = 15 - 4 = 11 \tag{12}$$

$$M_{32} = \det \begin{bmatrix} 2 & 1 \\ 1 & 5 \end{bmatrix} = 10 - 1 = 9 \tag{13}$$

$$M_{33} = \det \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} = 8 - 3 = 5 \tag{14}$$

The complete minor matrix is:

$$\mathbf{M} = \begin{bmatrix} -6 & -14 & -10 \\ 1 & -1 & -5 \\ 11 & 9 & 5 \end{bmatrix} \tag{15}$$

1.3 Cofactor of a Matrix

Definition 1.2 (Cofactor)**.** The cofactor C_{ij} of element a_{ij} in an $n \times n$ matrix $\mathbf{A}$ is the signed minor:

$$C_{ij} = (-1)^{i+j} M_{ij} \tag{16}$$

where M_{ij} is the minor of a_{ij}.

The sign pattern follows a checkerboard pattern:

$$\begin{bmatrix} + & - & + & - & \cdots \\ - & + & - & + & \cdots \\ + & - & + & - & \cdots \\ - & + & - & + & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix} \tag{17}$$

Example 1.2 (Computing Cofactors)**.** Using the same matrix from the previous example:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \tag{18}$$

Step 1: Apply the sign pattern to each minor:

$$C_{11} = (-1)^{1+1}M_{11} = (+1)(-6) = -6 \tag{19}$$
$$C_{12} = (-1)^{1+2}M_{12} = (-1)(-14) = 14 \tag{20}$$
$$C_{13} = (-1)^{1+3}M_{13} = (+1)(-10) = -10 \tag{21}$$
$$C_{21} = (-1)^{2+1}M_{21} = (-1)(1) = -1 \tag{22}$$
$$C_{22} = (-1)^{2+2}M_{22} = (+1)(-1) = -1 \tag{23}$$
$$C_{23} = (-1)^{2+3}M_{23} = (-1)(-5) = 5 \tag{24}$$
$$C_{31} = (-1)^{3+1}M_{31} = (+1)(11) = 11 \tag{25}$$
$$C_{32} = (-1)^{3+2}M_{32} = (-1)(9) = -9 \tag{26}$$
$$C_{33} = (-1)^{3+3}M_{33} = (+1)(5) = 5 \tag{27}$$

The cofactor matrix is:

$$\mathbf{C} = \begin{bmatrix} -6 & 14 & -10 \\ -1 & -1 & 5 \\ 11 & -9 & 5 \end{bmatrix} \tag{28}$$

1.4 Computing Determinant using Cofactor Expansion

Theorem 1.1 (Laplace Expansion)**.** The determinant of an $n \times n$ matrix $\mathbf{A}$ can be computed using cofactor expansion along any row i or column j:

Row expansion:

$$\det(\mathbf{A}) = \sum_{j=1}^{n} a_{ij}C_{ij} = \sum_{j=1}^{n} a_{ij}(-1)^{i+j}M_{ij} \tag{29}$$

Column expansion:

$$\det(\mathbf{A}) = \sum_{i=1}^{n} a_{ij}C_{ij} = \sum_{i=1}^{n} a_{ij}(-1)^{i+j}M_{ij} \tag{30}$$

Example 1.3 (Determinant Calculation using Cofactors)**.** For the matrix:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \tag{31}$$

Method 1: Expand along row 1:

$$\det(\mathbf{A}) = a_{11}C_{11} + a_{12}C_{12} + a_{13}C_{13} \tag{32}$$
$$= (2)(-6) + (3)(14) + (1)(-10) \tag{33}$$
$$= -12 + 42 - 10 \tag{34}$$
$$= 20 \tag{35}$$

Method 2: Expand along column 3 (verification):

$$\det(\mathbf{A}) = a_{13}C_{13} + a_{23}C_{23} + a_{33}C_{33} \tag{36}$$
$$= (1)(-10) + (5)(5) + (1)(5) \tag{37}$$
$$= -10 + 25 + 5 \tag{38}$$
$$= 20 \tag{39}$$

Both methods yield the same result: $\det(\mathbf{A}) = 20$

1.5 Applications in Machine Learning

ML/DL Applications

1. Matrix Invertibility: The determinant (computed using cofactors) determines if a matrix is invertible. In linear regression, we need $\det(\mathbf{X}^T\mathbf{X}) \neq 0$ to solve for weights.
2. Numerical Stability: Understanding minors helps in analyzing condition numbers and numerical stability of algorithms.
3. Theoretical Foundation: Cofactor expansion provides theoretical understanding of:

- Gaussian processes (covariance matrix properties)
- Eigenvalue computation methods
- Matrix decomposition algorithms

4. Computational Geometry: Minors are used in computing volumes and cross products in 3D space, relevant for computer vision applications.

Example 1.4 (ML Application: Checking Linear Independence)**.** In feature selection, we need to check if features are linearly independent. Consider feature matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 1 & 1 & 2 \end{bmatrix} \tag{40}$$

Computing determinant using cofactor expansion along row 1:

$$\det(\mathbf{X}) = 1 \cdot \det \begin{bmatrix} 4 & 5 \\ 1 & 2 \end{bmatrix} - 2 \cdot \det \begin{bmatrix} 2 & 5 \\ 1 & 2 \end{bmatrix} + 3 \cdot \det \begin{bmatrix} 2 & 4 \\ 1 & 1 \end{bmatrix} \tag{41}$$

$$= 1(8-5) - 2(4-5) + 3(2-4) \tag{42}$$

$$= 3 + 2 - 6 \tag{43}$$

$$= -1 \neq 0 \tag{44}$$

Since $\det(\mathbf{X}) \neq 0$, the features are linearly independent and all can be retained.

2 Adjoint of a Matrix

2.1 Definition and Concept

Definition 2.1 (Adjoint Matrix)**.** The adjoint (or adjugate) of a square matrix $\mathbf{A}$, denoted as $\text{adj}(\mathbf{A})$, is the transpose of the cofactor matrix $\mathbf{C}$:

$$\text{adj}(\mathbf{A}) = \mathbf{C}^T \tag{45}$$

where $\mathbf{C} = [C_{ij}]$ is the cofactor matrix.

Equivalently:

$$[\text{adj}(\mathbf{A})]_{ij} = C_{ji} \tag{46}$$

Property 2.1 (Key Properties of Adjoint)**.** For an $n \times n$ matrix $\mathbf{A}$:

1. $\mathbf{A} \cdot \text{adj}(\mathbf{A}) = \text{adj}(\mathbf{A}) \cdot \mathbf{A} = \det(\mathbf{A})\mathbf{I}$
2. $\det(\text{adj}(\mathbf{A})) = [\det(\mathbf{A})]^{n-1}$
3. $\text{adj}(\mathbf{A}^T) = [\text{adj}(\mathbf{A})]^T$
4. $\text{adj}(\mathbf{AB}) = \text{adj}(\mathbf{B}) \cdot \text{adj}(\mathbf{A})$
5. If $\det(\mathbf{A}) \neq 0$, then $\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\text{adj}(\mathbf{A})$

2.2 Step-by-Step Computation

Example 2.1 (Computing Adjoint Matrix)**.** Find the adjoint of:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \tag{47}$$

Step 1: Compute all cofactors
We already computed these in the previous section:

$$\mathbf{C} = \begin{bmatrix} -6 & 14 & -10 \\ -1 & -1 & 5 \\ 11 & -9 & 5 \end{bmatrix} \tag{48}$$

Step 2: Transpose the cofactor matrix

$$\text{adj}(\mathbf{A}) = \mathbf{C}^T = \begin{bmatrix} -6 & -1 & 11 \\ 14 & -1 & -9 \\ -10 & 5 & 5 \end{bmatrix} \tag{49}$$

Step 3: Verification
Verify the property $\mathbf{A} \cdot \text{adj}(\mathbf{A}) = \det(\mathbf{A})\mathbf{I}$:

$$\mathbf{A} \cdot \text{adj}(\mathbf{A}) = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} -6 & -1 & 11 \\ 14 & -1 & -9 \\ -10 & 5 & 5 \end{bmatrix} \tag{50}$$

Computing element (1,1):

$$(2)(-6) + (3)(14) + (1)(-10) = -12 + 42 - 10 = 20 \tag{51}$$

Computing element (1,2):

$$(2)(-1) + (3)(-1) + (1)(5) = -2 - 3 + 5 = 0 \tag{52}$$

Computing element (1,3):

$$(2)(11) + (3)(-9) + (1)(5) = 22 - 27 + 5 = 0 \tag{53}$$

Continuing this process:

$$\mathbf{A} \cdot \text{adj}(\mathbf{A}) = \begin{bmatrix} 20 & 0 & 0 \\ 0 & 20 & 0 \\ 0 & 0 & 20 \end{bmatrix} = 20\mathbf{I} \tag{54}$$

Since $\det(\mathbf{A}) = 20$, this confirms our calculation is correct!

2.3 Adjoint for 2×2 Matrices

Theorem 2.1 (Adjoint of 2×2 Matrix)**.** For a 2×2 matrix:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \tag{55}$$

The adjoint is:

$$\text{adj}(\mathbf{A}) = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \tag{56}$$

This is obtained by swapping the diagonal elements and negating the off-diagonal elements.

Example 2.2 (2×2 Adjoint Example)**.** For:

$$\mathbf{A} = \begin{bmatrix} 5 & 2 \\ 3 & 1 \end{bmatrix} \tag{57}$$

Direct formula:

$$\text{adj}(\mathbf{A}) = \begin{bmatrix} 1 & -2 \\ -3 & 5 \end{bmatrix} \tag{58}$$

Verification:

$$\mathbf{A} \cdot \text{adj}(\mathbf{A}) = \begin{bmatrix} 5 & 2 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -3 & 5 \end{bmatrix} \tag{59}$$

$$= \begin{bmatrix} 5-6 & -10+10 \\ 3-3 & -6+5 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \tag{60}$$

And $\det(\mathbf{A}) = 5 \cdot 1 - 2 \cdot 3 = -1$, so $\mathbf{A} \cdot \text{adj}(\mathbf{A}) = -1 \cdot \mathbf{I}$

2.4 Applications in Machine Learning

ML/DL Applications of Adjoint

1. Matrix Inversion: The adjoint provides a method to compute matrix inverse:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\text{adj}(\mathbf{A}) \tag{61}$$

This is crucial in solving linear systems in ML algorithms like linear regression.

2. Solving Systems of Linear Equations: For $\mathbf{Ax} = \mathbf{b}$:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{\det(\mathbf{A})}\text{adj}(\mathbf{A})\mathbf{b} \tag{62}$$

3. Cramer's Rule: Each component of solution can be expressed using determinants:

$$x_i = \frac{\det(\mathbf{A}_i)}{\det(\mathbf{A})} \tag{63}$$

where $\mathbf{A}_i$ is $\mathbf{A}$ with column i replaced by $\mathbf{b}$.

4. Sensitivity Analysis: The adjoint helps analyze how perturbations in data affect model parameters.

Example 2.3 (ML Application: Linear Regression using Adjoint)**.** Given training data:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \tag{64}$$

Find the optimal weights $\mathbf{w}$ using normal equation: $\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Step 1: Compute $\mathbf{X}^T\mathbf{X}$

$$\mathbf{X}^T\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \tag{65}$$

Step 2: Compute adjoint

$$\text{adj}(\mathbf{X}^T\mathbf{X}) = \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} \tag{66}$$

Step 3: Compute determinant

$$\det(\mathbf{X}^T\mathbf{X}) = 3 \cdot 14 - 6 \cdot 6 = 42 - 36 = 6 \tag{67}$$

Step 4: Compute inverse

$$(\mathbf{X}^T\mathbf{X})^{-1} = \frac{1}{6}\begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 7/3 & -1 \\ -1 & 1/2 \end{bmatrix} \tag{68}$$

Step 5: Compute $\mathbf{X}^T\mathbf{y}$

$$\mathbf{X}^T\mathbf{y} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}\begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} 11 \\ 25 \end{bmatrix} \tag{69}$$

Step 6: Compute weights

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{70}$$

$$= \begin{bmatrix} 7/3 & -1 \\ -1 & 1/2 \end{bmatrix} \begin{bmatrix} 11 \\ 25 \end{bmatrix} \tag{71}$$

$$= \begin{bmatrix} 77/3 - 25 \\ -11 + 25/2 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 3/2 \end{bmatrix} \tag{72}$$

The regression line is: $\hat{y} = \frac{2}{3} + \frac{3}{2}x$

3 Inverse of a Matrix

3.1 Definition and Properties

Definition 3.1 (Matrix Inverse)**.** For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the inverse matrix $\mathbf{A}^{-1}$ is defined as the unique matrix that satisfies:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \tag{73}$$

where $\mathbf{I}$ is the $n \times n$ identity matrix.

A matrix is called **invertible** or **non-singular** if its inverse exists.

Theorem 3.1 (Conditions for Invertibility)**.** A square matrix $\mathbf{A}$ is invertible if and only if:

1. $\det(\mathbf{A}) \neq 0$ (non-zero determinant)
2. $\text{rank}(\mathbf{A}) = n$ (full rank)
3. The columns (or rows) of $\mathbf{A}$ are linearly independent
4. The null space contains only the zero vector: $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$

Property 3.1 (Properties of Matrix Inverse)**.** For invertible matrices $\mathbf{A}$ and $\mathbf{B}$:

1. $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
2. $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ (order reverses!)
3. $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$
4. $(k\mathbf{A})^{-1} = \frac{1}{k}\mathbf{A}^{-1}$ for scalar $k \neq 0$
5. $\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})}$

3.2 Methods for Computing Inverse

3.2.1 Method 1: Using Adjoint (Analytical)

Theorem 3.2 (Inverse via Adjoint)**.** For an invertible matrix $\mathbf{A}$:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\text{adj}(\mathbf{A}) \tag{74}$$

Example 3.1 (Computing Inverse using Adjoint)**.** Find the inverse of:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \tag{75}$$

Step 1: We already found:

- $\det(\mathbf{A}) = 20$

- $\text{adj}(\mathbf{A}) = \begin{bmatrix} -6 & -1 & 11 \\ 14 & -1 & -9 \\ -10 & 5 & 5 \end{bmatrix}$

Step 2: Apply the formula:

$$\mathbf{A}^{-1} = \frac{1}{20} \begin{bmatrix} -6 & -1 & 11 \\ 14 & -1 & -9 \\ -10 & 5 & 5 \end{bmatrix} = \begin{bmatrix} -0.3 & -0.05 & 0.55 \\ 0.7 & -0.05 & -0.45 \\ -0.5 & 0.25 & 0.25 \end{bmatrix} \tag{76}$$

Step 3: Verification

$$\mathbf{A}\mathbf{A}^{-1} = \begin{bmatrix} 2 & 3 & 1 \\ 1 & 4 & 5 \\ 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} -0.3 & -0.05 & 0.55 \\ 0.7 & -0.05 & -0.45 \\ -0.5 & 0.25 & 0.25 \end{bmatrix} \tag{77}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I} \tag{78}$$

Thus, the inverse is verified correctly

3.2.2 Method 2: Gauss-Jordan Elimination

Theorem 3.3 (Inverse via Row Reduction)**.** To find $\mathbf{A}^{-1}$:

1. Form the augmented matrix $[\mathbf{A}|\mathbf{I}]$
2. Perform row operations to transform it to $[\mathbf{I}|\mathbf{A}^{-1}]$
3. The right side is the inverse

Example 3.2 (Gauss-Jordan Method)**.** Find the inverse of:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \tag{79}$$

Step 1: Form augmented matrix

$$[\mathbf{A}|\mathbf{I}] = \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{array}\right] \tag{80}$$

Step 2: Row operations
$R_2 \leftarrow R_2 - 3R_1$:

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & -2 & -3 & 1 \end{array}\right] \tag{81}$$

$R_2 \leftarrow -\frac{1}{2}R_2$:

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 1 & 3/2 & -1/2 \end{array}\right] \tag{82}$$

$R_1 \leftarrow R_1 - 2R_2$:

$$\left[\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & 1 & 3/2 & -1/2 \end{array}\right] \tag{83}$$

Result:

$$\mathbf{A}^{-1} = \begin{bmatrix} -2 & 1 \\ 3/2 & -1/2 \end{bmatrix} \tag{84}$$

Verification:

$$\mathbf{A}\mathbf{A}^{-1} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -2 & 1 \\ 3/2 & -1/2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{85}$$

3.2.3 Method 3: For 2×2 Matrices (Direct Formula)

Theorem 3.4 (2×2 Inverse Formula)**.** For:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \tag{86}$$

If $\det(\mathbf{A}) = ad - bc \neq 0$, then:

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \tag{87}$$

Example 3.3 (Quick 2×2 Inverse)**.** For:

$$\mathbf{A} = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \tag{88}$$

Step 1: Compute determinant

$$\det(\mathbf{A}) = (4)(6) - (7)(2) = 24 - 14 = 10 \tag{89}$$

Step 2: Apply formula

$$\mathbf{A}^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix} \tag{90}$$

3.3 Pseudo-Inverse (Moore-Penrose Inverse)

Definition 3.2 (Pseudo-Inverse)**.** For a non-square or singular matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, the Moore-Penrose pseudo-inverse $\mathbf{A}^+$ is the unique matrix satisfying:

1. $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$
2. $\mathbf{A}^+\mathbf{A}\mathbf{A}^+ = \mathbf{A}^+$
3. $(\mathbf{A}\mathbf{A}^+)^T = \mathbf{A}\mathbf{A}^+$ (symmetric)
4. $(\mathbf{A}^+\mathbf{A})^T = \mathbf{A}^+\mathbf{A}$ (symmetric)

Theorem 3.5 (Computing Pseudo-Inverse)**.** For matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$:
Case 1: If $m \geq n$ and $\text{rank}(\mathbf{A}) = n$ (full column rank):

$$\mathbf{A}^+ = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T \tag{91}$$

Case 2: If $m \leq n$ and $\text{rank}(\mathbf{A}) = m$ (full row rank):

$$\mathbf{A}^+ = \mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1} \tag{92}$$

General Case: Use Singular Value Decomposition (SVD):

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \implies \mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T \tag{93}$$

where Σ^+ is formed by taking reciprocals of non-zero singular values.

Example 3.4 (Pseudo-Inverse for Overdetermined System)**.** Consider:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \tag{94}$$

This is 3×2 with full column rank.

Step 1: Compute $\mathbf{A}^T\mathbf{A}$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \tag{95}$$

Step 2: Invert $\mathbf{A}^T\mathbf{A}$

$$\det(\mathbf{A}^T\mathbf{A}) = 4 - 1 = 3 \tag{96}$$

$$(\mathbf{A}^T\mathbf{A})^{-1} = \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \tag{97}$$

Step 3: Compute pseudo-inverse

$$\mathbf{A}^+ = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T \tag{98}$$

$$= \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \tag{99}$$

$$= \frac{1}{3} \begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix} \tag{100}$$

Verification:

$$\mathbf{A}^+\mathbf{A} = \frac{1}{3} \begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{I}_2 \tag{101}$$

3.4 Applications in Machine Learning and Deep Learning

Critical ML/DL Applications

1. Linear Regression: The normal equation for finding optimal weights:

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{102}$$

This requires computing the inverse of $\mathbf{X}^T\mathbf{X}$. If $\mathbf{X}$ is not full rank, we use the pseudo-inverse.

2. Ridge Regression (Regularized):

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \tag{103}$$

The term $\lambda\mathbf{I}$ ensures invertibility even when $\mathbf{X}^T\mathbf{X}$ is singular.

3. Solving Linear Systems: Many optimization problems in ML reduce to solving $\mathbf{Ax} = \mathbf{b}$:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} \tag{104}$$

4. Neural Network Weight Initialization: The pseudo-inverse can be used to initialize weights in certain architectures (e.g., Extreme Learning Machines):

$$\mathbf{W}_{out} = \mathbf{YH}^+ \tag{105}$$

where $\mathbf{H}$ is the hidden layer output.

5. Kalman Filtering: State estimation in dynamical systems uses matrix inverses:

$$\mathbf{K} = \mathbf{PH}^T(\mathbf{HPH}^T + \mathbf{R})^{-1} \tag{106}$$

6. Whitening Transformation: Decorrelating features using covariance matrix:

$$\mathbf{X}_{white} = \mathbf{\Sigma}^{-1/2}(\mathbf{X} - \boldsymbol{\mu}) \tag{107}$$

Example 3.5 (Complete ML Example: Multiple Linear Regression)**.** Given house price data:

$$\mathbf{X} = \begin{bmatrix} 1 & 1200 & 2 \\ 1 & 1500 & 3 \\ 1 & 1800 & 3 \\ 1 & 2000 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 200000 \\ 250000 \\ 280000 \\ 320000 \end{bmatrix} \tag{108}$$

Find weights $\mathbf{w} = [w_0, w_1, w_2]^T$ for: Price $= w_0 + w_1 \cdot \text{area} + w_2 \cdot \text{bedrooms}$

Step 1: Compute $\mathbf{X}^T\mathbf{X}$

$$\mathbf{X}^T\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1200 & 1500 & 1800 & 2000 \\ 2 & 3 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1200 & 2 \\ 1 & 1500 & 3 \\ 1 & 1800 & 3 \\ 1 & 2000 & 4 \end{bmatrix} \tag{109}$$

$$= \begin{bmatrix} 4 & 6500 & 12 \\ 6500 & 10890000 & 19800 \\ 12 & 19800 & 38 \end{bmatrix} \tag{110}$$

Step 2: Compute $\mathbf{X}^T\mathbf{y}$

$$\mathbf{X}^T\mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1200 & 1500 & 1800 & 2000 \\ 2 & 3 & 3 & 4 \end{bmatrix} \begin{bmatrix} 200000 \\ 250000 \\ 280000 \\ 320000 \end{bmatrix} = \begin{bmatrix} 1050000 \\ 1756000000 \\ 3230000 \end{bmatrix} \tag{111}$$

Step 3: Solve $(\mathbf{X}^T\mathbf{X})\mathbf{w} = \mathbf{X}^T\mathbf{y}$
Using computational tools (in practice, use numpy.linalg.solve):

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \approx \begin{bmatrix} 50000 \\ 100 \\ 20000 \end{bmatrix} \tag{112}$$

Interpretation:

- Base price: \$50,000
- Each sq ft adds: \$100
- Each bedroom adds: \$20,000

Prediction for new house (1600 sq ft, 3 bedrooms):

$$\text{Price} = 50000 + 100(1600) + 20000(3) = \$270,000 \tag{113}$$

3.5 Computational Considerations

Important Numerical Notes

1. **Avoid Direct Inversion:** Direct matrix inversion is:
   - Computationally expensive: $O(n^3)$
   - Numerically unstable for ill-conditioned matrices
2. **Better Alternatives:**
   - Use `numpy.linalg.solve(A, b)` instead of `np.dot(np.linalg.inv(A), b)`
   - Use LU, QR, or Cholesky decomposition
   - Use iterative methods for large systems
3. **Condition Number:** The condition number $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$ measures numerical stability:
   - $\kappa(\mathbf{A}) \approx 1$: well-conditioned
   - $\kappa(\mathbf{A}) \gg 1$: ill-conditioned (unstable inversion)
4. **Regularization:** For ill-conditioned $\mathbf{X}^T\mathbf{X}$, use ridge regression:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \tag{114}$$

where $\lambda > 0$ improves condition number.

4 Rank of a Matrix

4.1 Definition and Concepts

Definition 4.1 (Matrix Rank)**.** The rank of a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$, denoted $\text{rank}(\mathbf{A})$ or $r(\mathbf{A})$, is the maximum number of linearly independent rows (row rank) or columns (column rank) in the matrix.

Key fact: Row rank = Column rank (always equal)

$$\text{rank}(\mathbf{A}) = \dim(\text{Column Space}) = \dim(\text{Row Space}) \tag{115}$$

Theorem 4.1 (Properties of Rank)**.** For matrices $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$:

1. $0 \leq \text{rank}(\mathbf{A}) \leq \min(m, n)$
2. $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}^T)$
3. $\text{rank}(\mathbf{AB}) \leq \min(\text{rank}(\mathbf{A}), \text{rank}(\mathbf{B}))$
4. $\text{rank}(\mathbf{A} + \mathbf{B}) \leq \text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B})$
5. If $\mathbf{A}$ is $m \times n$ with rank r, then:
   - $\dim(\text{Null Space}) = n - r$ (Rank-Nullity Theorem)
   - $\mathbf{A}$ has r non-zero singular values

Definition 4.2 (Full Rank)**.** A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is:

- **Full column rank:** if $\text{rank}(\mathbf{A}) = n$ (columns are linearly independent)
- **Full row rank:** if $\text{rank}(\mathbf{A}) = m$ (rows are linearly independent)
- **Full rank:** if $\text{rank}(\mathbf{A}) = \min(m, n)$

For square matrices ($m = n$): Full rank $\iff$ $\det(\mathbf{A}) \neq 0$ $\iff$ invertible

4.2 Methods for Computing Rank

4.2.1 Method 1: Row Echelon Form (REF)

Theorem 4.2 (Rank from REF)**.** The rank of a matrix equals the number of non-zero rows in its row echelon form.

Example 4.1 (Computing Rank using REF)**.** Find the rank of:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 5 & 7 \end{bmatrix} \tag{116}$$

Step 1: Row reduction
$R_2 \leftarrow R_2 - 2R_1$:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 3 & 5 & 7 \end{bmatrix} \tag{117}$$

$R_3 \leftarrow R_3 - 3R_1$:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & -1 & -2 \end{bmatrix} \tag{118}$$

$R_2 \leftrightarrow R_3$ (swap rows):

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{bmatrix} \tag{119}$$

Result: 2 non-zero rows $\implies \text{rank}(\mathbf{A}) = 2$
Interpretation: The third row is a linear combination of the first two rows:

$$\text{Row } 3 = 3 \cdot \text{Row } 1 - 1 \cdot \text{Row } 2 \tag{120}$$

4.2.2 Method 2: Determinants of Submatrices

Theorem 4.3 (Rank via Minors)**.** The rank of $\mathbf{A}$ is the size of the largest square submatrix with non-zero determinant.

Example 4.2 (Rank using Determinants)**.** For:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 0 \end{bmatrix} \tag{121}$$

Check 3×3 determinant:

$$\det(\mathbf{A}) = 1 \cdot \det \begin{bmatrix} 1 & 4 \\ 0 & 0 \end{bmatrix} = 0 \tag{122}$$

Check 2×2 submatrix:

$$\det \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = 1 \neq 0 \tag{123}$$

Since we found a 2×2 submatrix with non-zero determinant but no 3×3, $\text{rank}(\mathbf{A}) = 2$.

4.2.3 Method 3: Singular Value Decomposition (SVD)

Theorem 4.4 (Rank from SVD)**.** For $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ (SVD):

$$\text{rank}(\mathbf{A}) = \text{number of non-zero singular values in } \mathbf{\Sigma} \tag{124}$$

Example 4.3 (Rank using SVD)**.** Consider:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \tag{125}$$

Computing SVD (numerically):

$$\mathbf{\Sigma} = \begin{bmatrix} 8.12 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \tag{126}$$

Only 1 non-zero singular value $\implies \text{rank}(\mathbf{A}) = 1$
Verification: Column 2 = 2 × Column 1, so columns are linearly dependent.

4.3 Applications in Machine Learning

ML/DL Applications of Rank

1. **Linear Regression Solvability:** For the system $\mathbf{Xw} = \mathbf{y}$:
   - If $\text{rank}(\mathbf{X}) = n$ (full column rank): unique solution via $\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$
   - If $\text{rank}(\mathbf{X}) < n$: underdetermined (use regularization or pseudo-inverse)
2. **Feature Selection and Multicollinearity:**
   - If $\text{rank}(\mathbf{X}) < n$ features, some features are redundant
   - Multicollinearity occurs when features are nearly linearly dependent
   - Use rank to detect and remove redundant features
3. **Neural Network Weight Matrices:**
   - Low-rank weight matrices indicate reduced model capacity
   - Rank can be used to compress neural networks
   - Low-rank factorization: $\mathbf{W} \approx \mathbf{UV}^T$ where $\mathbf{U} \in \mathbb{R}^{m\times r}$, $\mathbf{V} \in \mathbb{R}^{n\times r}$ with $r \ll \min(m, n)$
4. **Dimensionality Reduction:**
   - PCA finds rank-k approximation of data
   - Truncated SVD keeps top k singular values
   - Effective rank: $r_{eff} = \frac{(\sum_i \sigma_i)^2}{\sum_i \sigma_i^2}$
5. **Matrix Factorization in Recommender Systems:**
   - User-item matrix $\mathbf{R} \in \mathbb{R}^{m\times n}$ is typically low-rank
   - Factorize: $\mathbf{R} \approx \mathbf{UV}^T$ with rank $k \ll \min(m, n)$
   - Lower rank = stronger generalization

Example 4.4 (ML Application: Detecting Multicollinearity)**.** Given feature matrix for predicting house prices:

$$\mathbf{X} = \begin{bmatrix} 1000 & 500 & 2000 \\ 1500 & 750 & 3000 \\ 2000 & 1000 & 4000 \\ 2500 & 1250 & 5000 \end{bmatrix} \tag{127}$$

Features: [Area in sq ft, Half of area, Area in sq meters ($\times$ 2)]
Step 1: Check rank
Notice that:

- Column 2 = 0.5 $\times$ Column 1

- Column 3 = 2 × Column 1

Step 2: Compute rank using REF
After row reduction:

$$\text{REF}(\mathbf{X}) = \begin{bmatrix} 1000 & 500 & 2000 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{128}$$

$\text{rank}(\mathbf{X}) = 1$ (only 1 independent feature!)
Step 3: Solution
To fix multicollinearity, keep only one feature:

$$\mathbf{X}_{reduced} = \begin{bmatrix} 1000 \\ 1500 \\ 2000 \\ 2500 \end{bmatrix} \tag{129}$$

Now $\text{rank}(\mathbf{X}_{reduced}) = 1 = n$ (full column rank for single feature).
Impact:

- Original: $\mathbf{X}^T\mathbf{X}$ is singular (non-invertible)
- Reduced: $\mathbf{X}_{reduced}^T\mathbf{X}_{reduced}$ is invertible
- Can now solve normal equations without regularization

Example 4.5 (Deep Learning Application: Low-Rank Neural Network Compression)**.** Consider a fully connected layer with weight matrix $\mathbf{W} \in \mathbb{R}^{512\times1024}$.
Original computation:

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \text{Parameters: } 512 \times 1024 = 524,288 \tag{130}$$

Low-rank approximation (rank $r = 64$):
Factorize $\mathbf{W} \approx \mathbf{U}\mathbf{V}^T$ where:

- $\mathbf{U} \in \mathbb{R}^{512\times64}$
- $\mathbf{V} \in \mathbb{R}^{1024\times64}$

Compressed computation:

$$\mathbf{y} = \mathbf{U}(\mathbf{V}^T\mathbf{x}) \tag{131}$$

Parameters: $512 \times 64 + 1024 \times 64 = 98,304$
Compression ratio: $\frac{524,288}{98,304} \approx 5.3\times$ fewer parameters!
Trade-off: Slight accuracy loss for significant memory and computation savings.

4.4 Rank-Nullity Theorem

Theorem 4.5 (Rank-Nullity Theorem)**.** For a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ represented by matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$$\text{rank}(\mathbf{A}) + \dim(\text{Null}(\mathbf{A})) = n \tag{132}$$

where:

- $\text{rank}(\mathbf{A})$ = dimension of column space (range)
- $\dim(\text{Null}(\mathbf{A}))$ = dimension of null space (kernel)
- n = number of columns (input dimension)

Example 4.6 (Rank-Nullity Application)**.** For:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix} \tag{133}$$

Find rank:
Row reduction gives:

$$\text{REF}(\mathbf{A}) = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \end{bmatrix} \tag{134}$$

$\text{rank}(\mathbf{A}) = 1$
Find nullity:
By Rank-Nullity: $\dim(\text{Null}(\mathbf{A})) = 3 - 1 = 2$
Find null space basis:
Solve $\mathbf{Ax} = \mathbf{0}$:

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{135}$$

From row 1: $x_1 + 2x_2 + 3x_3 = 0 \implies x_1 = -2x_2 - 3x_3$
Let $x_2 = s$ and $x_3 = t$:

$$\mathbf{x} = \begin{bmatrix} -2s - 3t \\ s \\ t \end{bmatrix} = s \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} + t \begin{bmatrix} -3 \\ 0 \\ 1 \end{bmatrix} \tag{136}$$

Null space basis:

$$\text{Null}(\mathbf{A}) = \text{span} \left\{ \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -3 \\ 0 \\ 1 \end{bmatrix} \right\} \tag{137}$$

Dimension = 2 (confirms Rank-Nullity)

5 Column Space and Null Space

5.1 Column Space (Range)

Definition 5.1 (Column Space)**.** The column space (or range) of matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, denoted $\mathcal{C}(\mathbf{A})$ or $\text{Col}(\mathbf{A})$, is the set of all possible linear combinations of the columns of $\mathbf{A}$:

$$\mathcal{C}(\mathbf{A}) = \{\mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} = \text{span}\{\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_n\} \tag{138}$$

where $\mathbf{a}_i$ are the columns of $\mathbf{A}$.

Properties:

- $\mathcal{C}(\mathbf{A})$ is a subspace of $\mathbb{R}^m$
- $\dim(\mathcal{C}(\mathbf{A})) = \text{rank}(\mathbf{A})$
- $\mathbf{b} \in \mathcal{C}(\mathbf{A}) \iff \mathbf{A}\mathbf{x} = \mathbf{b}$ has a solution

Example 5.1 (Finding Column Space)**.** For:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 5 & 7 \end{bmatrix} \tag{139}$$

Step 1: Identify columns

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}, \quad \mathbf{a}_3 = \begin{bmatrix} 3 \\ 6 \\ 7 \end{bmatrix} \tag{140}$$

Step 2: Check linear independence
Notice: $\mathbf{a}_3 = \mathbf{a}_1 + \mathbf{a}_2$

$$\begin{bmatrix} 3 \\ 6 \\ 7 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 2 \\ 4 \\ 4 \end{bmatrix} \quad \text{(approximately)} \tag{141}$$

Actually, checking: $2\mathbf{a}_1 = \mathbf{a}_3 - \mathbf{a}_2$... Let's use row reduction.
Step 3: Row reduction to find pivot columns

$$\text{RREF}(\mathbf{A}) = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 1 & 2 \end{bmatrix} \to \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{bmatrix} \tag{142}$$

Pivot columns: 1 and 2
Step 4: Column space basis

$$\mathcal{C}(\mathbf{A}) = \text{span}\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \right\} \tag{143}$$

$\dim(\mathcal{C}(\mathbf{A})) = 2 = \text{rank}(\mathbf{A})$

5.2 Null Space (Kernel)

Definition 5.2 (Null Space)**.** The null space (or kernel) of matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, denoted $\mathcal{N}(\mathbf{A})$ or $\text{Null}(\mathbf{A})$, is the set of all vectors that are mapped to zero:

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\} \tag{144}$$

Properties:

- $\mathcal{N}(\mathbf{A})$ is a subspace of $\mathbb{R}^n$
- $\dim(\mathcal{N}(\mathbf{A})) = n - \text{rank}(\mathbf{A})$ (Rank-Nullity)
- $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\} \iff \mathbf{A}$ has full column rank

Example 5.2 (Finding Null Space)**.** For:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \end{bmatrix} \tag{145}$$

Step 1: Solve $\mathbf{A}\mathbf{x} = \mathbf{0}$
Row reduction:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \end{bmatrix} \to \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{146}$$

Step 2: Express in parametric form
From row 1: $x_1 + 2x_2 + 3x_3 + 4x_4 = 0$

$$x_1 = -2x_2 - 3x_3 - 4x_4 \tag{147}$$

Let $x_2 = s$, $x_3 = t$, $x_4 = u$ (free variables):

$$\mathbf{x} = \begin{bmatrix} -2s - 3t - 4u \\ s \\ t \\ u \end{bmatrix} = s \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix} + t \begin{bmatrix} -3 \\ 0 \\ 1 \\ 0 \end{bmatrix} + u \begin{bmatrix} -4 \\ 0 \\ 0 \\ 1 \end{bmatrix} \tag{148}$$

Step 3: Null space basis

$$\mathcal{N}(\mathbf{A}) = \text{span}\left\{ \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} -3 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -4 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\} \tag{149}$$

Verification:

- $\text{rank}(\mathbf{A}) = 1$ (one non-zero row)
- $\dim(\mathcal{N}(\mathbf{A})) = 3$
- $1 + 3 = 4 = n$ (Rank-Nullity)

5.3 Geometric Interpretation

Geometric Understanding

Column Space:

- Represents all possible outputs of the linear transformation $\mathbf{Ax}$
- A hyperplane (or subspace) in output space $\mathbb{R}^m$
- For $\mathbf{b}$ to be solvable in $\mathbf{Ax} = \mathbf{b}$, we need $\mathbf{b} \in \mathcal{C}(\mathbf{A})$

Null Space:

- Contains all vectors that "disappear" under transformation
- Represents directions of information loss
- If $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$, the transformation is injective (one-to-one)

Orthogonality:

$$\mathcal{C}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A}) \tag{150}$$

The row space is orthogonal to the null space.

5.4 Applications in Machine Learning

ML/DL Applications

1. Solving Linear Systems (Linear Regression):
For $\mathbf{Xw} = \mathbf{y}$:

- Solution exists $\iff$ $\mathbf{y} \in \mathcal{C}(\mathbf{X})$
- If $\mathbf{y} \notin \mathcal{C}(\mathbf{X})$, find least squares solution
- Multiple solutions exist if $\mathcal{N}(\mathbf{X}) \neq \{\mathbf{0}\}$

2. Feature Redundancy:

- If $\dim(\mathcal{N}(\mathbf{X})) > 0$, features are redundant
- Null space vectors show linear dependencies among features
- Use to identify and remove correlated features

3. Principal Component Analysis (PCA):

- PCA finds basis for column space of centered data
- Components span $\mathcal{C}(\mathbf{X})$
- Discarded components span approximate null space (low variance directions)

4. Neural Network Analysis:

- Null space of weight matrix = inputs that don't affect output
- Understanding null space helps in:
  - Adversarial robustness
  - Feature importance
  - Network pruning

Example 5.3 (ML Application: Understanding Linear Regression Solutions)**.** Given overdetermined system (more equations than unknowns):

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 3 \\ 5 \\ 7 \\ 9 \end{bmatrix} \tag{151}$$

Question: Does exact solution exist?
Step 1: Find column space basis

The columns are linearly independent (different values), so:

$$\mathcal{C}(\mathbf{X}) = \text{span}\left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} \right\} \tag{152}$$

This is a 2D subspace of $\mathbb{R}^4$.
Step 2: Check if $\mathbf{y} \in \mathcal{C}(\mathbf{X})$
We need to check if $\mathbf{y}$ can be written as:

$$\mathbf{y} = w_0 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + w_1 \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} \tag{153}$$

From first equation: $3 = w_0 + 2w_1$ From second equation: $5 = w_0 + 3w_1$
Subtracting: $w_1 = 2$, so $w_0 = -1$
Check with other equations:

$$\text{Row 3: } -1 + 2(4) = 7\checkmark \tag{154}$$
$$\text{Row 4: } -1 + 2(5) = 9\checkmark \tag{155}$$

Conclusion: $\mathbf{y} \in \mathcal{C}(\mathbf{X})$, so exact solution exists!

$$\mathbf{w} = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \tag{156}$$

The regression line is: $\hat{y} = -1 + 2x$
Null space: Since $\text{rank}(\mathbf{X}) = 2 = n$, we have $\mathcal{N}(\mathbf{X}) = \{\mathbf{0}\}$, so the solution is unique.

Example 5.4 (Feature Selection using Null Space)**.** Feature matrix with redundancy:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 & 5 \\ 2 & 4 & 6 & 10 \\ 3 & 6 & 9 & 15 \end{bmatrix} \tag{157}$$

Features: $[x_1, x_2 = 2x_1, x_3 = 3x_1, x_4 = x_1 + x_3]$
Step 1: Find null space
Row reduction shows rank = 1, so nullity = 3.
Solving $\mathbf{Xw} = \mathbf{0}$:

$$\mathcal{N}(\mathbf{X}) = \text{span}\left\{ \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} -3 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -4 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\} \tag{158}$$

Step 2: Interpret dependencies
First null vector: $-2x_1 + x_2 = 0 \implies x_2 = 2x_1$ Second null vector: $-3x_1 + x_3 = 0 \implies x_3 = 3x_1$ Third null vector: $-4x_1 + x_4 = 0 \implies x_4 = 4x_1$
Step 3: Feature selection
Keep only x_1 (or any single feature). Remove x_2, x_3, x_4 as they're redundant.
Result: Reduced from 4 features to 1, eliminating multicollinearity.

6 Change of Basis

6.1 Introduction and Motivation

Definition 6.1 (Basis)**.** A basis for a vector space $\mathbb{R}^n$ is a set of n linearly independent vectors $\{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$ that span $\mathbb{R}^n$.

Any vector $\mathbf{v} \in \mathbb{R}^n$ can be uniquely expressed as:

$$\mathbf{v} = c_1\mathbf{b}_1 + c_2\mathbf{b}_2 + \cdots + c_n\mathbf{b}_n \tag{159}$$

The coefficients $[c_1, c_2, \ldots, c_n]^T$ are called the coordinates of $\mathbf{v}$ in the basis $\{\mathbf{b}_i\}$.

Definition 6.2 (Standard Basis)**.** The standard basis for $\mathbb{R}^n$ consists of unit vectors:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \ldots, \quad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} \tag{160}$$

6.2 Change of Basis Matrix

Definition 6.3 (Change of Basis)**.** Let $\mathcal{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$ be a basis for $\mathbb{R}^n$. The change of basis matrix is:

$$\mathbf{P}_{\mathcal{B}} = [\mathbf{b}_1|\mathbf{b}_2|\cdots|\mathbf{b}_n] \tag{161}$$

Coordinate transformations:

$$[\mathbf{v}]_{\mathcal{B}} = \mathbf{P}_{\mathcal{B}}^{-1}\mathbf{v} \quad (\text{standard to } \mathcal{B}) \tag{162}$$

$$\mathbf{v} = \mathbf{P}_{\mathcal{B}}[\mathbf{v}]_{\mathcal{B}} \quad (\mathcal{B} \text{ to standard}) \tag{163}$$

where $[\mathbf{v}]_{\mathcal{B}}$ denotes coordinates of $\mathbf{v}$ in basis $\mathcal{B}$.

Example 6.1 (Change of Basis in $\mathbb{R}^2$)**.** Consider vector $\mathbf{v} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$ in standard basis.

New basis:

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \tag{164}$$

Step 1: Form change of basis matrix

$$\mathbf{P} = [\mathbf{b}_1|\mathbf{b}_2] = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{165}$$

Step 2: Find inverse

$$\mathbf{P}^{-1} = \frac{1}{-2}\begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \tag{166}$$

Step 3: Transform to new basis

$$[\mathbf{v}]_{\mathcal{B}} = \mathbf{P}^{-1}\mathbf{v} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}\begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 5/2 \\ 1/2 \end{bmatrix} \tag{167}$$

Verification:

$$\frac{5}{2}\mathbf{b}_1 + \frac{1}{2}\mathbf{b}_2 = \frac{5}{2}\begin{bmatrix} 1 \\ 1 \end{bmatrix} + \frac{1}{2}\begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \mathbf{v}\checkmark \tag{168}$$

6.3 Matrix Representation in Different Bases

Theorem 6.1 (Matrix Change of Basis)**.** Let $\mathbf{A}$ be a linear transformation represented in the standard basis. In basis $\mathcal{B}$ with change of basis matrix $\mathbf{P}$, the transformation is represented as:

$$[\mathbf{A}]_{\mathcal{B}} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P} \tag{169}$$

This is called a similarity transformation.

Example 6.2 (Matrix in Different Basis)**.** Given transformation matrix:

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \tag{170}$$

Express in basis $\mathcal{B} = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\}$

Step 1: We already have:

$$\mathbf{P} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{P}^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \tag{171}$$

Step 2: Compute similarity transformation

$$[\mathbf{A}]_{\mathcal{B}} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P} \tag{172}$$

$$= \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{173}$$

First: $\mathbf{A}\mathbf{P} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 3 & -1 \end{bmatrix}$

Then:

$$[\mathbf{A}]_{\mathcal{B}} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 3 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \tag{174}$$

Result: In the new basis, $\mathbf{A}$ becomes diagonal!

$$[\mathbf{A}]_{\mathcal{B}} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \tag{175}$$

This is the eigenvalue decomposition - the basis vectors are eigenvectors with eigenvalues 3 and 1.

6.4 Orthonormal Bases

Definition 6.4 (Orthonormal Basis)**.** A basis $\{\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n\}$ is orthonormal if:

$$\mathbf{q}_i^T \mathbf{q}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \tag{176}$$

The change of basis matrix $\mathbf{Q} = [\mathbf{q}_1 | \mathbf{q}_2 | \cdots | \mathbf{q}_n]$ satisfies:

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{I} \implies \mathbf{Q}^{-1} = \mathbf{Q}^T \tag{177}$$

Advantage: Computing $\mathbf{Q}^{-1}$ is just transposition (very efficient)!

Example 6.3 (Orthonormal Basis in $\mathbb{R}^3$)**.** Standard orthonormal basis:

$$\mathbf{q}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{q}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{q}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{178}$$

Another orthonormal basis (rotated 45° in xy-plane):

$$\mathbf{q}_1' = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}, \quad \mathbf{q}_2' = \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}, \quad \mathbf{q}_3' = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{179}$$

Verification of orthonormality:

$$\mathbf{q}_1'^T \mathbf{q}_1' = \frac{1}{2} + \frac{1}{2} + 0 = 1\checkmark \tag{180}$$

$$\mathbf{q}_1'^T \mathbf{q}_2' = -\frac{1}{2} + \frac{1}{2} + 0 = 0\checkmark \tag{181}$$

$$\mathbf{q}_1'^T \mathbf{q}_3' = 0 + 0 + 0 = 0\checkmark \tag{182}$$

Change of basis matrix:

$$\mathbf{Q} = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{183}$$

Inverse (just transpose):

$$\mathbf{Q}^{-1} = \mathbf{Q}^T = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ -1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{184}$$

6.5 Applications in Machine Learning and Deep Learning

ML/DL Applications of Change of Basis

1. Principal Component Analysis (PCA):
PCA transforms data to a new basis where:

- Basis vectors = principal components (eigenvectors of covariance matrix)
- New coordinates = uncorrelated features
- Maximum variance along first few axes

Process:

$$\mathbf{Z} = \mathbf{XV} = \mathbf{XP} \tag{185}$$

where $\mathbf{V}$ contains eigenvectors as columns.

2. Feature Space Transformations:

- Whitening: Transform to basis where covariance = identity
- Decorrelation: Remove correlations between features
- Standardization: Zero mean, unit variance in each dimension

3. Kernel Methods (Implicit Change of Basis):
SVMs use kernels to implicitly map data to high-dimensional feature space:

$$\phi : \mathbb{R}^n \to \mathbb{R}^m \quad (m \gg n) \tag{186}$$

without explicitly computing the transformation.

4. Word Embeddings (NLP):
Change from one-hot encoding to dense embeddings:

- Original: sparse basis (vocabulary size)
- Embedding: dense basis (e.g., 300 dimensions)
- Learned transformation captures semantic relationships

5. Visualization:
Projecting high-dimensional data to 2D/3D for visualization:

- t-SNE: nonlinear change of basis
- UMAP: preserves local and global structure
- PCA: linear projection to principal components

Example 6.4 (Complete PCA Example with Change of Basis)**.** Dataset (centered):

$$\mathbf{X} = \begin{bmatrix} -1 & -1 \\ -1 & 1 \\ 1 & -1 \\ 1 & 1 \end{bmatrix} \tag{187}$$

Step 1: Compute covariance matrix

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} = \frac{1}{3}\begin{bmatrix}-1 & -1 & 1 & 1\\ -1 & 1 & -1 & 1\end{bmatrix}\begin{bmatrix}-1 & -1\\ -1 & 1\\ 1 & -1\\ 1 & 1\end{bmatrix} = \frac{1}{3}\begin{bmatrix}4 & 0\\ 0 & 4\end{bmatrix} = \begin{bmatrix}4/3 & 0\\ 0 & 4/3\end{bmatrix} \tag{188}$$

Step 2: Find eigenvectors
Since $\mathbf{C}$ is already diagonal, eigenvectors are standard basis:

$$\mathbf{v}_1 = \begin{bmatrix}1\\0\end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix}0\\1\end{bmatrix} \tag{189}$$

with eigenvalues $\lambda_1 = \lambda_2 = 4/3$.
Step 3: Change of basis matrix

$$\mathbf{P} = [\mathbf{v}_1|\mathbf{v}_2] = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} = \mathbf{I} \tag{190}$$

Result: Data already aligned with principal components (no rotation needed).
More interesting example - rotated data:

$$\mathbf{X}' = \begin{bmatrix}-\sqrt{2} & 0\\ 0 & \sqrt{2}\\ \sqrt{2} & 0\\ 0 & -\sqrt{2}\end{bmatrix} \tag{191}$$

Covariance:

$$\mathbf{C}' = \frac{1}{3}\begin{bmatrix}4 & 0\\ 0 & 4\end{bmatrix} \tag{192}$$

Still diagonal! The variance is equal in all directions (circular distribution).
For correlated data:

$$\mathbf{X}'' = \begin{bmatrix}1 & 1\\ 2 & 2\\ 3 & 3\\ 4 & 4\end{bmatrix} - \begin{bmatrix}2.5 & 2.5\\ 2.5 & 2.5\\ 2.5 & 2.5\\ 2.5 & 2.5\end{bmatrix} = \begin{bmatrix}-1.5 & -1.5\\ -0.5 & -0.5\\ 0.5 & 0.5\\ 1.5 & 1.5\end{bmatrix} \tag{193}$$

Covariance:

$$\mathbf{C}'' = \frac{1}{3}\begin{bmatrix}5 & 5\\ 5 & 5\end{bmatrix} \tag{194}$$

Eigenvectors:

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix} \tag{195}$$

Eigenvalues: $\lambda_1 = 10/3$, $\lambda_2 = 0$
Principal component transformation:

$$\mathbf{P} = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix} \tag{196}$$

Transformed data:

$$\mathbf{Z} = \mathbf{X}''\mathbf{P} = \begin{bmatrix} -1.5 & -1.5 \\ -0.5 & -0.5 \\ 0.5 & 0.5 \\ 1.5 & 1.5 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{197}$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} -3 & 0 \\ -1 & 0 \\ 1 & 0 \\ 3 & 0 \end{bmatrix} \tag{198}$$

Interpretation:

- First PC: captures all variance (perfect correlation)
- Second PC: zero variance (redundant)
- Can reduce to 1D: keep only first column

Example 6.5 (Deep Learning Application: Layer Whitening)**.** In neural networks, whitening transforms inputs to have zero mean and identity covariance.

Input features:

$$\mathbf{X} = \begin{bmatrix} 2 & 4 \\ 3 & 6 \\ 4 & 8 \\ 5 & 10 \end{bmatrix} \tag{199}$$

Step 1: Center data

$$\boldsymbol{\mu} = \begin{bmatrix} 3.5 \\ 7 \end{bmatrix} \tag{200}$$

$$\mathbf{X}_{centered} = \begin{bmatrix} -1.5 & -3 \\ -0.5 & -1 \\ 0.5 & 1 \\ 1.5 & 3 \end{bmatrix} \tag{201}$$

Step 2: Compute covariance

$$\boldsymbol{\Sigma} = \frac{1}{3}\mathbf{X}_{centered}^T\mathbf{X}_{centered} = \begin{bmatrix} 1.667 & 3.333 \\ 3.333 & 6.667 \end{bmatrix} \tag{202}$$

Step 3: Eigendecomposition

$$\boldsymbol{\Sigma} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^T \tag{203}$$

Eigenvalues: $\lambda_1 = 8.334$, $\lambda_2 \approx 0$
Eigenvectors:

$$\mathbf{Q} = \begin{bmatrix} 0.447 & -0.894 \\ 0.894 & 0.447 \end{bmatrix} \tag{204}$$

Step 4: Whitening transformation

$$\mathbf{X}_{white} = \mathbf{X}_{centered}\mathbf{Q}\boldsymbol{\Lambda}^{-1/2}\mathbf{Q}^T \tag{205}$$

Or more simply:

$$\mathbf{X}_{white} = \mathbf{X}_{centered}\boldsymbol{\Sigma}^{-1/2} \tag{206}$$

Result: $\text{Cov}(\mathbf{X}_{white}) = \mathbf{I}$ (identity covariance)
Benefits in deep learning:

- Faster convergence during training
- Prevents gradient vanishing/exploding
- Reduces internal covariate shift
- Used in Batch Normalization

6.6 Summary and Key Takeaways

Key Concepts Summary

Change of Basis Essentials:

1. **Purpose:** Express same vector/transformation in different coordinate system
2. **Matrix:** $\mathbf{P} = [\mathbf{b}_1|\mathbf{b}_2|\cdots|\mathbf{b}_n]$
3. **Forward:** $\mathbf{v} = \mathbf{P}[\mathbf{v}]_{\mathcal{B}}$ (basis $\to$ standard)
4. **Backward:** $[\mathbf{v}]_{\mathcal{B}} = \mathbf{P}^{-1}\mathbf{v}$ (standard $\to$ basis)
5. **Transformation:** $[\mathbf{A}]_{\mathcal{B}} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$

Special Case - Orthonormal Basis:

- $\mathbf{Q}^{-1} = \mathbf{Q}^T$ (transpose = inverse)
- Computationally efficient
- Preserves lengths and angles
- Used in PCA, SVD, QR decomposition

ML/DL Applications:

- PCA: Transform to principal component basis
- Whitening: Transform to decorrelated basis
- Feature engineering: Domain-specific transformations
- Dimensionality reduction: Project to lower-dimensional basis

7 Conclusion

This document covered advanced matrix concepts essential for machine learning and deep learning:

1. **Minor and Cofactor:** Foundation for computing determinants and inverses
2. **Adjoint:** Used in analytical matrix inversion

3. **Matrix Inverse:** Critical for solving linear systems, pseudo-inverse for non-square matrices
4. **Matrix Rank:** Determines solvability and feature independence
5. **Column Space and Null Space:** Understanding linear transformations and redundancy
6. **Change of Basis:** Transforming data to more convenient coordinate systems

7.1 Practical Recommendations

Best Practices

For Implementation:

- Use `numpy.linalg.solve()` instead of explicit inversion
- Check condition number before inverting matrices
- Use SVD for pseudo-inverse computation
- Apply regularization (Ridge) for ill-conditioned systems
- Use QR or Cholesky decomposition when appropriate

For Analysis:

- Check rank to detect multicollinearity
- Examine null space for feature redundancy
- Use eigendecomposition to understand transformations
- Apply PCA for dimensionality reduction
- Visualize data in principal component basis

7.2 Further Study

To deepen understanding:

- Study tensor operations for deep learning
- Learn automatic differentiation frameworks
- Explore matrix calculus in detail
- Understand numerical linear algebra
- Practice implementing algorithms from scratch